

Relatório 18 - Prática: Docker e Containers para Aplicações (III)

Igor Carvalho Marchi

1. Introdução

O objetivo desta prática foi compreender a utilização da interface de linha de comando do Docker para criar, executar e administrar containers e imagens. Durante a aula, foram apresentados conceitos relacionados ao ciclo de vida dos containers, mapeamento de portas, volumes, gerenciamento de imagens e construção de imagens por meio de Dockerfiles. A prática também permitiu compreender como o Docker pode facilitar a criação de ambientes isolados e reproduzíveis para aplicações.

2. Desenvolvimento

2.1 A interface de linha de comando do Docker

A prática iniciou com uma introdução à Docker CLI, que permite interagir com o Docker Engine por meio do terminal. A estrutura dos comandos apresentada segue o padrão `docker {objeto} {ação}`, em que o objeto indica o recurso que será manipulado e a ação determina o que será realizado. Entre os objetos apresentados estão `container`, `image` e `volume`, cada um possuindo comandos específicos para gerenciamento.

Um dos comandos apresentados foi o `docker container --help`, utilizado para consultar as ações disponíveis para containers. Esse tipo de consulta é importante porque permite verificar quais operações podem ser realizadas e quais opções podem ser utilizadas em cada comando.

2.2 Ciclo de vida dos containers

Um dos principais pontos da prática foi compreender o ciclo de vida de um container. Inicialmente, foi apresentado o comando `docker container create`, utilizado para criar um container a partir de uma imagem. No exemplo apresentado durante a aula, foi utilizada a imagem Alpine. Depois da criação, o container existe, mas não necessariamente está executando.

Para verificar os containers em execução, pode ser utilizado o `docker container ls`. Já o `docker container ls -a` permite visualizar todos os containers, incluindo aqueles que estão parados. Essa diferença permite compreender que um container pode ser criado e armazenado no ambiente mesmo sem estar sendo executado.

Depois da criação, o container pode ser iniciado utilizando `docker container start`. Para acessar o processo principal do container, foi apresentado o `docker container attach`.

Durante essa interação, foi possível observar que o comando `exit` encerra o processo principal e faz com que o container pare. Entretanto, os arquivos criados anteriormente permanecem no container enquanto ele não for removido. Dessa forma, ao iniciar novamente o container, os arquivos continuam disponíveis.

Também foram apresentados comandos para outras operações durante o ciclo de vida dos containers. O `docker container stop` permite parar um container, enquanto o `docker container rm` é utilizado para removê-lo. O `docker container rename` permite alterar o nome de um container existente.

2.3 Execução e gerenciamento de containers

Depois dos testes com containers interativos, foi demonstrada a execução de um servidor Nginx utilizando o comando `docker run -d --name my-nginx nginx`. A opção `-d` permite que o container seja executado em segundo plano, liberando o terminal enquanto o serviço continua funcionando.

Para acompanhar o funcionamento do container, foram apresentados comandos como `docker container logs` e `docker container inspect`. O primeiro permite consultar as informações registradas pelo processo em execução, enquanto o segundo apresenta informações sobre o próprio container. Também foi apresentado o `docker container exec`, que permite executar um comando diretamente dentro de um container sem a necessidade de entrar nele primeiro.

Outro comando estudado foi o `docker container cp`, utilizado para copiar arquivos ou diretórios entre o computador e o container. Dessa maneira, além de executar comandos dentro do ambiente isolado, também é possível realizar operações de gerenciamento entre o sistema do computador e o container.

Categoria	Comando	Função
Containers	<code>docker container ls</code>	Lista os containers em execução
Containers	<code>docker container create</code>	Cria um container
Containers	<code>docker container start</code>	Inicia um container
Containers	<code>docker container stop</code>	Para um container
Containers	<code>docker container exec</code>	Executa um comando dentro do container
Containers	<code>docker container logs</code>	Exibe os logs do container
Containers	<code>docker container inspect</code>	Exibe informações do container

Categoria	Comando	Função
Imagens	docker image ls	Lista as imagens disponíveis
Imagens	docker container commit	Cria uma imagem a partir de um container
Imagens	docker image save / load	Salva e carrega imagens
Volumes	docker volume create	Cria um volume
Sistema	docker system prune	Remove dados não utilizados

2.4 Mapeamento de portas

Por padrão, o container possui um ambiente isolado, inclusive em relação à rede. Por esse motivo, quando um serviço está sendo executado dentro do container, é necessário realizar o mapeamento de portas para permitir que ele seja acessado externamente.

Para demonstrar esse conceito, foi apresentada a utilização da opção -p, seguindo o formato:

docker run -p 8080:80 nginx



Nesse caso, a porta 8080 do computador é associada à porta 80 do container. Como o Nginx utiliza a porta 80 dentro do container, essa configuração permite direcionar o acesso realizado externamente para o serviço que está sendo executado no ambiente Docker.

2.5 Volumes e persistência dos dados

Outro conceito apresentado foi o mapeamento de volumes. Os containers possuem seu próprio sistema de arquivos e, quando um container é removido, os dados que dependem exclusivamente dele podem ser perdidos. Os volumes são utilizados para manter os dados separados do ciclo de vida do container.

O funcionamento apresentado durante a aula mostra o volume como uma forma de manter os dados armazenados na máquina e disponibilizá-los para o container. Dessa maneira, alterações realizadas nos arquivos podem ser mantidas mesmo quando o container é removido. Isso permite que os dados tenham uma persistência maior do que aquela proporcionada pelo armazenamento interno do container.

Para criar um volume, foi apresentado o comando `docker volume create`, que prepara o recurso que poderá posteriormente ser utilizado por containers.

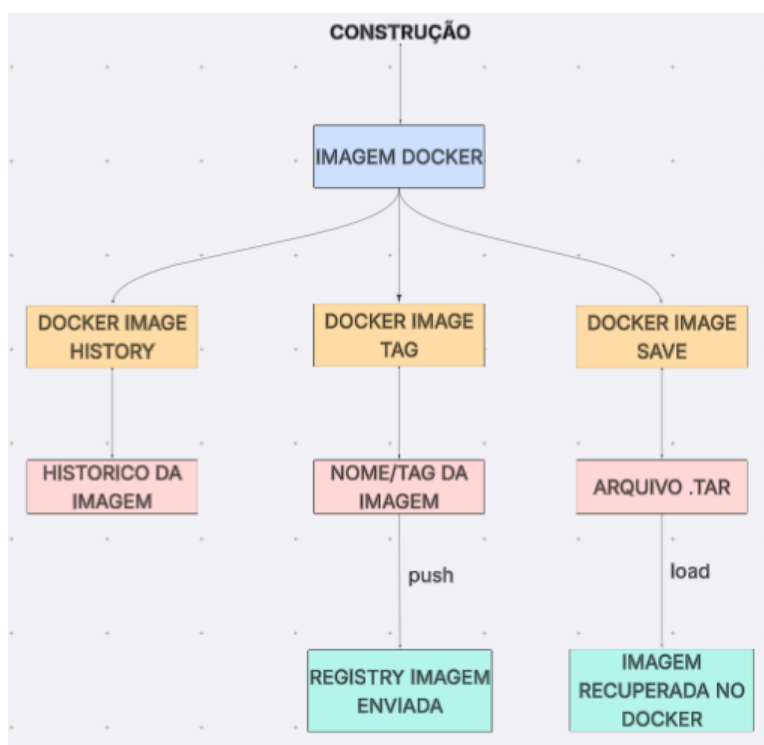
2.6 Imagens Docker

Depois do estudo dos containers, a prática passou para o gerenciamento das imagens. A imagem representa a base utilizada para criar containers. Durante a aula, foi demonstrado que, depois de realizar alterações em um container, é possível utilizar o comando `docker container commit` para transformar aquele estado em uma nova imagem.

Após o commit, a imagem pode ser verificada utilizando `docker image ls`. Esse processo mostra uma relação importante entre container e imagem: o container pode ser modificado durante sua execução e, posteriormente, essas modificações podem ser utilizadas para gerar uma nova imagem.

Também foram apresentados recursos para transportar imagens por meio de arquivos. O comando `docker image save` permite salvar uma ou mais imagens em um arquivo `.tar`. Depois disso, a imagem pode ser removida e recuperada utilizando `docker image load`. Dessa forma, foi possível compreender como uma imagem pode ser exportada, armazenada e posteriormente importada novamente para o Docker.

O comando `docker image history` também foi apresentado para consultar o histórico da imagem e visualizar as etapas relacionadas à sua construção. Além disso, foram abordados `docker image tag`, utilizado para criar uma tag para uma imagem, e `docker image push`, utilizado para enviar uma imagem para um registro.



2.7 Dockerfile e construção de imagens

A prática também apresentou os Dockerfiles como uma maneira de automatizar a construção de imagens. Em vez de criar um container e realizar todas as configurações manualmente, as instruções necessárias podem ser registradas em um arquivo chamado Dockerfile.

No exemplo apresentado, a instrução FROM foi utilizada para definir a imagem base, enquanto RUN foi utilizada para executar comandos durante a construção da imagem. Depois que o Dockerfile é criado, o comando de build pode ser utilizado para gerar uma imagem personalizada a partir das instruções definidas no arquivo.

Essa abordagem facilita a reprodução do ambiente porque as configurações deixam de depender apenas de ações manuais. O processo fica registrado em um arquivo que pode ser utilizado novamente para construir a imagem.

2.8 Docker System

Por fim, foram apresentados comandos relacionados à administração do ambiente Docker. O `docker system info` permite consultar informações do sistema Docker, enquanto o `docker system df` apresenta informações relacionadas ao espaço utilizado pelos recursos.

Também foram apresentados comandos relacionados ao `prune`, utilizados para remover dados que não estão sendo utilizados. Essa parte da prática mostrou que, além de criar e executar containers e imagens, também é necessário administrar os recursos armazenados no ambiente para evitar o acúmulo de dados desnecessários.

3. Conclusão

A prática permitiu compreender como a Docker CLI é utilizada para controlar os principais recursos do Docker. O estudo do ciclo de vida mostrou as diferentes etapas pelas quais um container passa, desde sua criação até sua execução, parada e remoção. O uso de portas mostrou como disponibilizar um serviço executado dentro de um ambiente isolado, enquanto os volumes apresentaram uma maneira de preservar os dados. O gerenciamento de imagens demonstrou recursos para criar, consultar, salvar e recuperar imagens, e o Dockerfile apresentou uma forma de automatizar sua construção. Dessa maneira, foi possível compreender como containers, imagens, volumes, portas e comandos de administração se relacionam dentro do ambiente Docker.

REFERÊNCIAS

Insights visuais -

https://lucid.app/lucidspark/18066335-de30-4eed-9880-54abe681d706/edit?invitationId=inv_21f49f9f-9f89-43de-80e8-b16ebb14e097&page=3B0CRSot8otl#

Aulas assistidas -

https://docs.google.com/document/d/1rAvJ0FiWZG5fgcej4VhHqDJcgQWh_O5CrmZCw812GGU/edit?tab=t.0#heading=h.ktzojxydotho

O que é container - <https://www.docker.com/resources/what-container/>